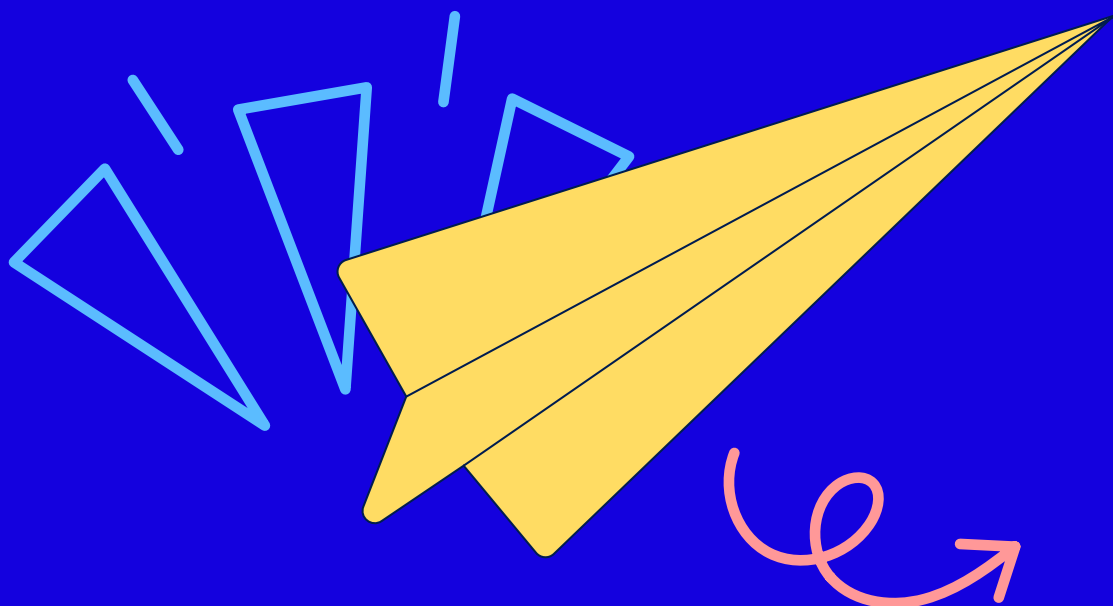




Introducción a la maquetación II: CSS

Nivelación



Introducción a la maquetación II: CSS

¿Qué es un sitio web sin un diseño atractivo, sin una experiencia de usuario optimizada? Todo sitio web que se quiera construir debe empezar por un proceso de UX/UI, es decir, un análisis de la experiencia que se le ofrecer al usuario y la construcción de una interfaz que atraiga y facilite la comprensión del mensaje que le queremos transmitir.

Después de esto, necesitamos construir el sitio web siguiendo estas guías marcadas por ese análisis y, para ello, el supoder del que disponemos es CSS. Los estilos de un sitio web marcan sin duda la diferencia y logran que este pueda gozar de mayor o menor éxito.

Autor: Miguel Ángel Muñoz



Herramientas necesarias



Selectores



Modelo de caja



Variables



Dimensiones



Posicionamiento



Layout



¿Qué hemos aprendido?

Herramientas necesarias



Para estudiar esta unidad didáctica y abordar su parte práctica debes acceder a nuestra herramienta amiga: Visual Studio Code.

CSS (*cascading style sheets*) es el **lenguaje que utilizamos para dar estilos**, no únicamente a HTML, también podemos estilar otros lenguajes como SVG.

Como suelo recomendar con HTML, te recomiendo aquí que también revises y estudies la documentación de Mozilla: es muy completa y está muy bien redactada. En este caso, puedes encontrarla en developer.mozilla.org/en-US/docs/Learn/CSS.

La estructura es sencilla, creamos un archivo con extensión '.css' y comenzamos a definir nuestros estilos.

En CSS debemos comprender estos **dos conceptos clave**:

Cascada —

Controla qué regla se aplica cuando existen conflictos (veremos un ejemplo en el siguiente apartado de selectores).

Herencia —

Hay propiedades que heredan su valor del padre, en caso de que no se especifique uno nuevo, por ejemplo, el color y todo lo relacionado con la fuente.

También es importante saber que si una propiedad no se ha definido correctamente no se producirá ningún error; y esto es bueno y malo, aunque más malo que bueno.

Es bueno porque **no deja de funcionar el sitio web** si cometemos un error sin darnos cuenta, pero es malo porque es más complicado detectarlo.

Selectores

Todo estilo lo tenemos que **asignar a un elemento HTML** (selector), incluso podemos asignarlo al propio elemento html:

```
html {  
  font-size: var(--font-size-global);  
  color: var(--white-color);  
  font-family: "VR_Standard", sans-serif;  
}
```

Vamos definiendo nuestras reglas de estilos siempre teniendo en cuenta que el funcionamiento es en forma de cascada, es decir, el último **estilo de la cascada** es el que se aplica en caso de que haya duplicidad o el estilo que se especifique en el selector más específico. Este sería un ejemplo:

```
section {  
  color: blue  
}  
  
section {  
  font-size: 2rem;  
}  
  
section.my-class {  
  font-size: 1rem;  
}
```

Aquí tenemos repetido el elemento `section`, aunque definiendo **dos propiedades distintas**, por lo que se van a aplicar ambas. En el caso de que el segundo `section` definiese otro color, este sería el que se aplicaría.

El tercer `section` tiene una modificación en el selector, se ha especificado en el selector que el `section` tenga una clase `my-class`. Por tanto, al ser **más específico** y siempre y cuando el elemento `section` tenga la clase `my-clases` definida, se aplicará la propiedad `font-size` con valor `1rem` en lugar de `2rem`.

Si el tercer selector, en lugar de tener `.my-class` todo junto, contuviese un espacio entre medias, **el selector haría referencia a un elemento hijo de `section`** que tuviese la clase `my-class`. En este caso, lo que estuviese en el `section` aplicaría un `font-size` de `2rem` y lo que estuviera dentro de un elemento con la clase `my-class`, bajo el elemento `section`, aplicaría un `font-size` de `1rem`.

Como este ejemplo hay múltiples casuísticas que podéis investigar en la documentación y que son fundamentales para ser capaz de definir todos los estilos necesarios.

A parte de los **selectores** de elemento, clase o ID, tenemos los siguientes:

Selectores de atributo

`a[title].`

Pseudoclases y pseudoelementos

`button:hover`, `button:before`, `li:first-child`.

Combinadores

`div > p`, que selecciona los párrafos que son hijos directos de un `div`.

Selector universal

Se representa con un `*` y sirve para seleccionar todos los elementos del documento. Normalmente, se utiliza para clarificar ciertos selectores que pueden crear confusión, por ejemplo.

Pero en el ejemplo anterior con los `section`, si quisiéramos seleccionar un elemento que tuviese la clase `my-class` en los elementos hijos, en lugar de una sección con esta clase, la sintaxis sería añadir un espacio: **`section .my-class`**. Esto puede crear confusión y no darnos cuenta de que está separado, por lo tanto, sería más intuitivo poner: **`section *.my-class`**.

Propiedades

Las propiedades son lo que **definen los estilos como tal**, lo que está dentro del bloque del elemento:

```
font-size: 1.6rem;
padding: 1rem 2rem 1rem 2rem;
border: 1px solid var(--white-color);
background-color: transparent;
color: var(--white-color);
cursor: pointer;
display: flex;
align-items: center;
flex-flow: row nowrap;
```

Miguel Ángel Muñoz Viejo,

Modelo de caja

Todo en CSS tiene una caja a su alrededor y el comportamiento de estas cajas lo definimos por medio de las propiedades de CSS. Un ejemplo de las cajas y de su comportamiento lo podemos ver entre el elemento 'spam' y el elemento 'p'.

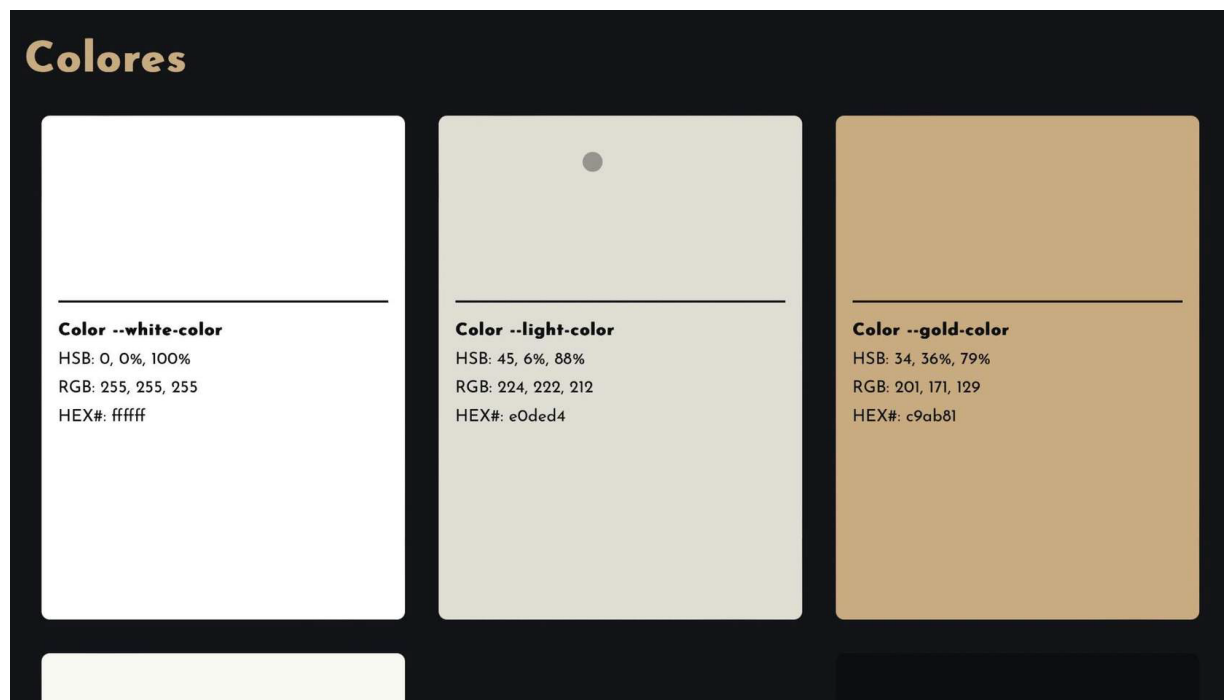
- Si incluimos un **elemento 'p'** nos daremos cuenta de que se añade una especie de bloque invisible que desplaza a otros elementos y provoca un salto de línea (modelo de caja externa block).
- Si incluimos un **elemento 'spam'** dentro del texto del elemento 'p', no notamos ningún efecto (modelo de caja externa inline).

Este comportamiento lo podemos cambiar a través de la **propiedad display**:

```
button {
  font-size: 1.6rem;
  padding: 1rem 2rem 1rem 2rem;
  border: 1px solid var(--white-color);
  background-color: transparent;
  color: var(--white-color);
  cursor: pointer;
  display: flex;
  align-items: center;
  flex-flow: row nowrap;
}

button:before {
  content: "";
  display: block;
  opacity: 0;
  width: 0;
  height: 1px;
  margin-right: 1rem;
  background-color: var(--white-color);
  transition: all 0.2s linear 0s;
}
```


Un ejemplo, donde seguro nos pelearemos con las cajas y su disposición, es cuando queramos disponer elementos unos al lado de otros, siendo bloques, o dividir en múltiples columnas una misma fila. Veamos este caso:



Aquí queremos distribuir los colores en filas de 3 columnas y, para ello, tendremos que aplicar reglas de layout que veremos más adelante.

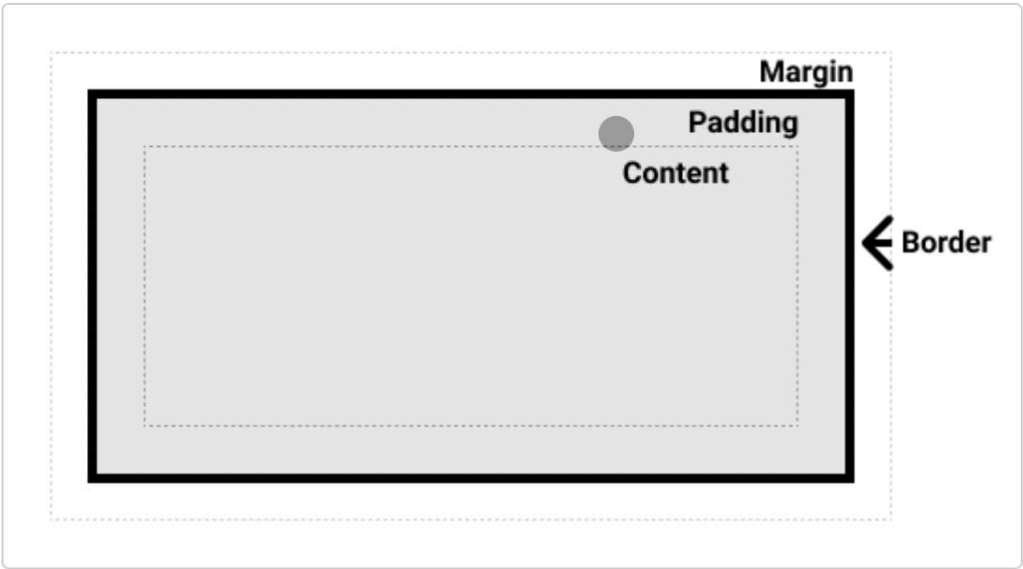
Por defecto cada uno de los div (en este caso) tiende a extenderse y a ocupar un width del 100% y a provocar un salto de línea para que el siguiente elemento se coloque por debajo, siguiendo el modelo de caja.

El modelo de caja tiene dos visualizaciones:

- **Externa:** que ya la hemos mencionado con los ejemplos de block e inline.
- **Interna:** que define cómo se comportan los elementos dentro del propio elemento, por ejemplo, con el modelo de caja flex.

Si añadimos a un div la propiedad display con el valor flex, la visualización externa será de tipo block, pero la visualización interna será de tipo flex, que sigue las reglas de flexbox (esto lo estudiaremos en el apartado dedicado al layout).

Las cajas se dividen en múltiples capas, al igual que una cebolla, ¡fíjate!



CONTENIDO	PADDING (RELLENO)	BORDE	MARGEN
La capa a la que afectan las propiedades de <i>width</i> y <i>height</i> .			

CONTENIDO	PADDING (RELLENO)	BORDE	MARGEN
El espacio alrededor del contenido y se establece por medio de la propiedad <i>padding</i> .			

CONTENIDO	PADDING (RELLENO)	BORDE	MARGEN
Envuelve el contenido y el relleno de la caja y se controla por medio de la propiedad <i>border</i> .			

CONTENIDO	PADDING (RELLENO)	BORDE	MARGEN
La capa más externa, define el espacio alrededor de todo lo demás y se controla por medio de la propiedad <i>margin</i> .			

Asimilar estas capas de la caja de los elementos es fundamental, ya que se tiende a usar indistintamente el *padding* y el *margin*, por ejemplo, pero no son lo mismo. También es importante cuando estamos jugando con disposiciones de cajas y anchos de estas: si queremos crear 4 columnas y establecemos un *width* a cada una de un 25% pero le añadimos un relleno, un borde o un margen, no estaremos consiguiendo el resultado deseado.

También tiene afectación en la definición de la propiedad `background`, que no debemos olvidar que afecta hasta la capa del relleno y no se mostrará en la capa de margen.

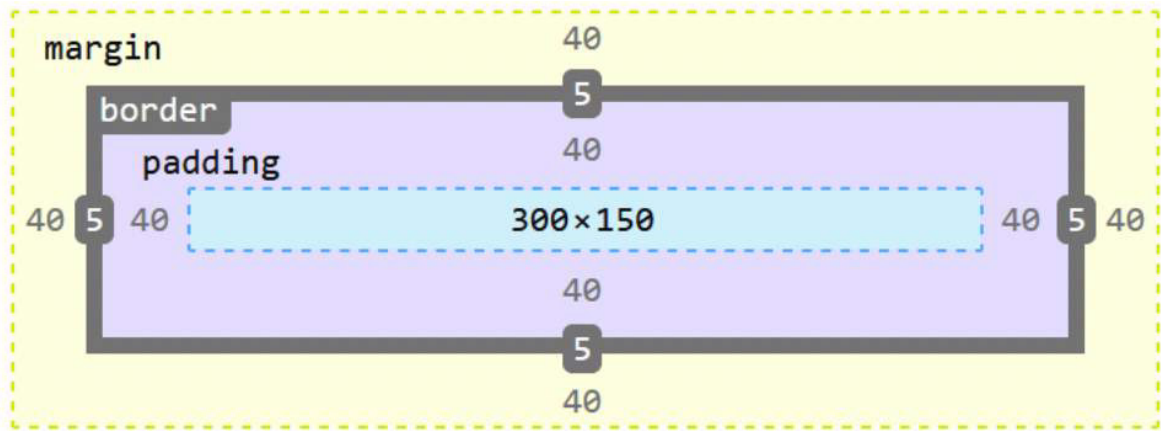
- Este comportamiento es el modelo de caja estándar, pero existe un modelo de caja alternativo en el que se incluye tanto el relleno como el borde dentro del ancho y alto del elemento, definidos por medio de las propiedades *width* y *height*.
- Este comportamiento se controla por medio de la propiedad *box-sizing*, ajustando un valor de *border-box*, que como se deduce de su nombre establece la caja hasta el borde.

Para establecer este **comportamiento alternativo** a nivel global, echa un vistazo al código.

```
html {  
  box-sizing: border-box;  
}  
*,  
*::before,  
*::after {  
  box-sizing: inherit;  
}
```

En cualquier caso, no uses este modo en el ejercicio práctico, ya que es interesante que trabajéis con el estándar para que entendáis las implicaciones.

Si quieres ver el modelo de cajas de un elemento de una página web, lo puedes hacer a través de las DevTools del navegador, seleccionando el elemento.



Variables

CSS nos permite el uso de variables, a través de las cuales podemos definir valores de propiedades, por ejemplo:

```
html {  
  --font-size-global: 16px;  
  --min-width: 320px;  
  
  /* COLORS */  
  --white-color: white;  
  --white-color-rgb: #ffffff;  
  --white-color-hsb: 0, 0%, 100%;  
  --light-color: rgb(224, 222, 212);  
  --light-color-rgb: #e0ded4;  
  --light-color-hsb: 45, 6%, 88%;  
  --gold-color: rgb(201, 171, 129);  
  --gold-color-rgb: #c9ab81;  
  --gold-color-hsb: 34, 36%, 79%;  
  --beige-color: rgb(247, 248, 241);  
  --beige-color-rgb: #f7f8f1;  
  --beige-color-hsb: 60, 3%, 97%;  
  --dark-color: rgb(18, 20, 23);  
  --dark-color-rgb: #121417;  
  --dark-color-hsb: 220, 14%, 8%;  
  --extra-dark-color: rgb(12, 14, 16);  
  --extra-dark-color-rgb: #0c0e10;  
  --extra-dark-color-hsb: 220, 25%, 6%;  
  
  /* LAYOUT */  
  --breackpoint-sm: 576px;  
  --breackpoint-md: 768px;  
  --breackpoint-lg: 992px;  
  --breackpoint-xl: 1200px;  
  --breackpoint-xxl: 1400px;  
}
```

Su uso es sencillo, se declaran añadiendo dos guiones (--) al comienzo junto con el nombre que le queramos asignar a la variable y se le asigna un valor de propiedad; y se utilizan por medio de la función **var()**.

```
html {  
  font-size: var(--font-size-global);  
  color: var(--white-color);  
  font-family: "VR_Standard", sans-serif;  
}
```

El selector bajo el que las definamos determina el ámbito de aplicación (*scope*), por lo tanto, si las definimos bajo un selector de párrafo 'p' únicamente aplicarán bajo este tipo de elemento. Si queremos que el ámbito de aplicación sea global, podemos usar el selector html o la pseudoclase **:root**.

Puede pasar que vayamos a utilizar una variable y que esta no exista, para evitar estos casos podemos usar lo que se denomina fallback.

Si falla existe esta alternativa:

```
.tres {  
  background-color: var(  
    --my-var,  
    var(--my-background, pink)  
  ); /* Rosa (pink) si my-var y --my-background no están definidas */  
}  
  
.tres {  
  background-color: var(  
    --my-var,  
    --my-background,  
    pink  
  ); /* Invalido: "--background, pink" */  
}
```

El segundo ejemplo es incorrecto, ya que no se pueden concatenar nombres de variables. Es necesario utilizarlas con la función var().

Import

El uso de variables nos lleva a tener una hoja de estilos que destinemos a la definición de las variables globales del sitio para que sean **utilizadas por el resto de las hojas de estilo**. Por ello es importante que veamos la importación. Imaginemos que tenemos una hoja de estilos con todas las variables a la que llamamos variables.css y otra hoja de estilos con los estilos globales del sitio que llamamos global.css.

Queremos **importar las variables** en la hoja de estilos global para poder usarla:

<> guide.html	# global.css X	# variables.css	# guide.css
---------------	----------------	-----------------	-------------

```
css > # global.css >  button
1  @import url("../variables.css");
2  @import url("../layout.css");
3  @import url("../colors.css");
```


Dimensiones

A la hora de definir dimensiones para los elementos, lo más habitual es utilizar **pixels**:

```
h4, .header4 {  
  font-size: 24px;  
  font-weight: 400;  
  padding: 1rem 0;  
}
```

Pero también disponemos de otras opciones que en función de para qué pueden ser más adecuadas.

Em

Establece una unidad de medida **relativa al tamaño de letra establecida en el navegador que, por defecto, son 16px**, aunque este valor puede ser modificado por el propio usuario a diferencia de la unidad de medida px.

También podemos **establecer un font-size** base en el elemento html para que utilice este como referencia, en lugar del establecido por el navegador:

```
html {  
  font-size: var(--font-size-global);  
  color: var(--white-color);  
  font-family: "VR_Standard", sans-serif;  
}
```

La **peculiaridad** que tiene la unidad de medida em es que depende del último elemento padre que haya definido un *font-size*, por ejemplo:

```
.guide-content {  
  font-size: 1.6em;  
  float: left;  
  width: 33%;  
}  
  
.guide-content__container {  
  font-size: 0.9em;  
  margin: 1rem;  
  padding: 1rem;  
  border-radius: 0.5rem;  
  background-color: var(--extra-dark-color);  
  height: 27rem;  
}
```

La clase *.guide-content* establece un *font-size* de 1.6em, es decir, $1,6 * 16\text{px} = 25,6\text{px}$ y la clase *.guide-content__container* establece un *font-size* de 0,9em, es decir, $25,6\text{px} * 0,9 = 23,04\text{px}$, ya que esta es hija de la anterior.

Rem

Al igual que la unidad de medida em, rem toma como base o bien el *font-size* definido por el navegador, o bien el definido en el elemento HTML. A diferencia de *em*, no se tienen en cuenta otras definiciones de *font-size*, **únicamente la establecida en la raíz**, de ahí el origen del nombre *Root em*. Por tanto, en el ejemplo anterior, la clase *.guide-content* tendrá un valor de fuente de $16\text{px} * 1,6$ y la clase *.guide-content__container* un valor de $16\text{px} * 0,9$ y no tiene en cuenta el valor establecido por el padre.

%

Las medidas en porcentaje están más **orientadas a anchos de caja** para poder obtener dimensiones más flexibles en función del tamaño del elemento padre. Puede establecer 2 columnas iguales asignando un 50% de ancho a cada elemento (no olvides tener en cuenta el modelo de caja) o desiguales asignando, por ejemplo, un 30% y un 70%.

VW y VH

Estas **unidades de medida tienen una finalidad muy similar al porcentaje**, pero con una salvedad. Si te has dado cuenta, en la definición de la unidad de medida %, he indicado que se establecen en función del tamaño del elemento padre, es decir, si el elemento padre mide el 50% del *viewport*, entonces las medidas se establecen en función de ese tamaño.

Por otro lado, estas medidas vw y vh, utilizan como referencia el tamaño del *viewport*, de ahí sus nombres, *viewport's width* y *viewport's height*. Por tanto, si el *viewport* mide 400px y establecemos un *width* de 100vw, tendremos esos 400px y lo mismo para el alto.

Posicionamiento

Con posicionamiento me refiero a la **posición que toma un elemento dentro del documento web** y a las reglas que nos permiten alterar el posicionamiento estándar de los elementos.

Static

Es el **comportamiento estándar**, por lo tanto, nos es útil cuando queramos crear unas clases con una serie de propiedades y queramos tener en cuenta los diferentes tipos de posicionamiento.

Relative

Es muy similar al posicionamiento anterior ya que, si únicamente indicamos la propiedad *position: relative*, se comportará exactamente igual que *static*. La diferencia viene cuando usamos las propiedades de posicionamiento (*top, right, bottom, left*) que sí que le afectarán y se posicionará en base a las medidas que indiquemos tomando como referencia la posición en la que estaba de origen.

Estas unidades de posicionamiento indican a la distancia que se tiene que separar con respecto a la posición del elemento.

Por ejemplo:

```
.element-moved {  
    position: relative;  
    top: 1rem;  
    left: 1rem;  
}
```

En este caso, desplazará el elemento 16px hacia abajo y 16px hacia la derecha, ya que se está indicando que se quiere dejar un espacio de 16px en la parte superior y otro espacio de 16px en la parte izquierda.

Absolute

Este tipo de posicionamiento provoca que el elemento salga del flujo normal del documento y se coloque en **una nueva capa separada del resto**. Podéis pensar, por ejemplo, en una modal que se abre encima del contenido principal: no está en el flujo normal, está en otra capa por encima.

Su posicionamiento **se controla exactamente igual**, a través de las propiedades de posicionamiento, la diferencia es que la referencia de posicionamiento no es la posición original del elemento, es la del elemento que lo contiene.

Z-index

Con esta propiedad añadimos la **tercera dimensión**. Con el tipo de posicionamiento y las propiedades de posicionamiento podemos mover el elemento en dos dimensiones 'x' e 'y'. Cuando tenemos varios elementos que los hemos posicionado uno encima de otro, esta propiedad es la que nos ayuda a establecer la dimensión 'z', es decir, definir qué elemento está por encima de otro.

Cuanto más grande sea el valor de la propiedad *z-index*, más arriba en el eje 'z' estará y, por tanto, estará por encima.

El valor únicamente puede ser un número, no se utilizan unidades como *px*, *em* o *rem*.

Fixed

El tipo de posicionamiento *fixed* funciona exactamente igual que el *absolute* con la salvedad de que mientras en **el posicionamiento *absolute* la posición es relativa al ancestro más cercano**, en el posicionamiento *fixed* la posición es relativa a la parte visible del *viewport*, con la excepción de que exista otro elemento padre con un posicionamiento *fixed*.

Un ejemplo para el **uso del posicionamiento *fixed* sería el menú del sitio web**. Si queremos que esté todo el tiempo visible en la parte superior, por mucho que hagas scroll el elemento *fixed* siempre será visible y estará colocado en la misma posición.

Sticky

Es una mezcla entre el posicionamiento *relative* y el posicionamiento *fixed*, ya que mantiene un comportamiento *relative* hasta que se hace visible en el *viewport* y toma la posición que tendría como elemento *fixed*.

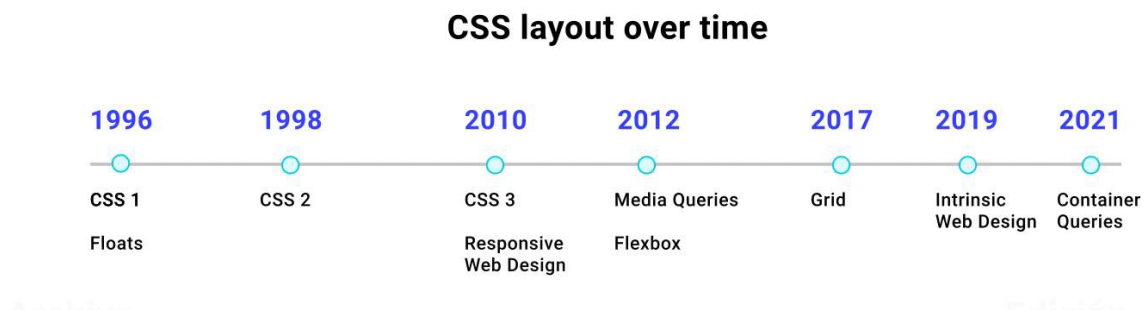


Podéis ver ejemplos de cada una de las opciones e información más ampliada [aquí](#).

Layout

El layout es lo que **define la disposición de los elementos** en nuestro documento web y para ello disponemos de diferentes técnicas.

Recomiendo que antes de nada hagas una prueba creando un archivo HTML y añadas diferentes elementos y observes cómo se comportan y disponen a lo largo del documento. Así, paralelamente, puedes crear otros archivos HTML y hacer pruebas con los siguientes flujos.

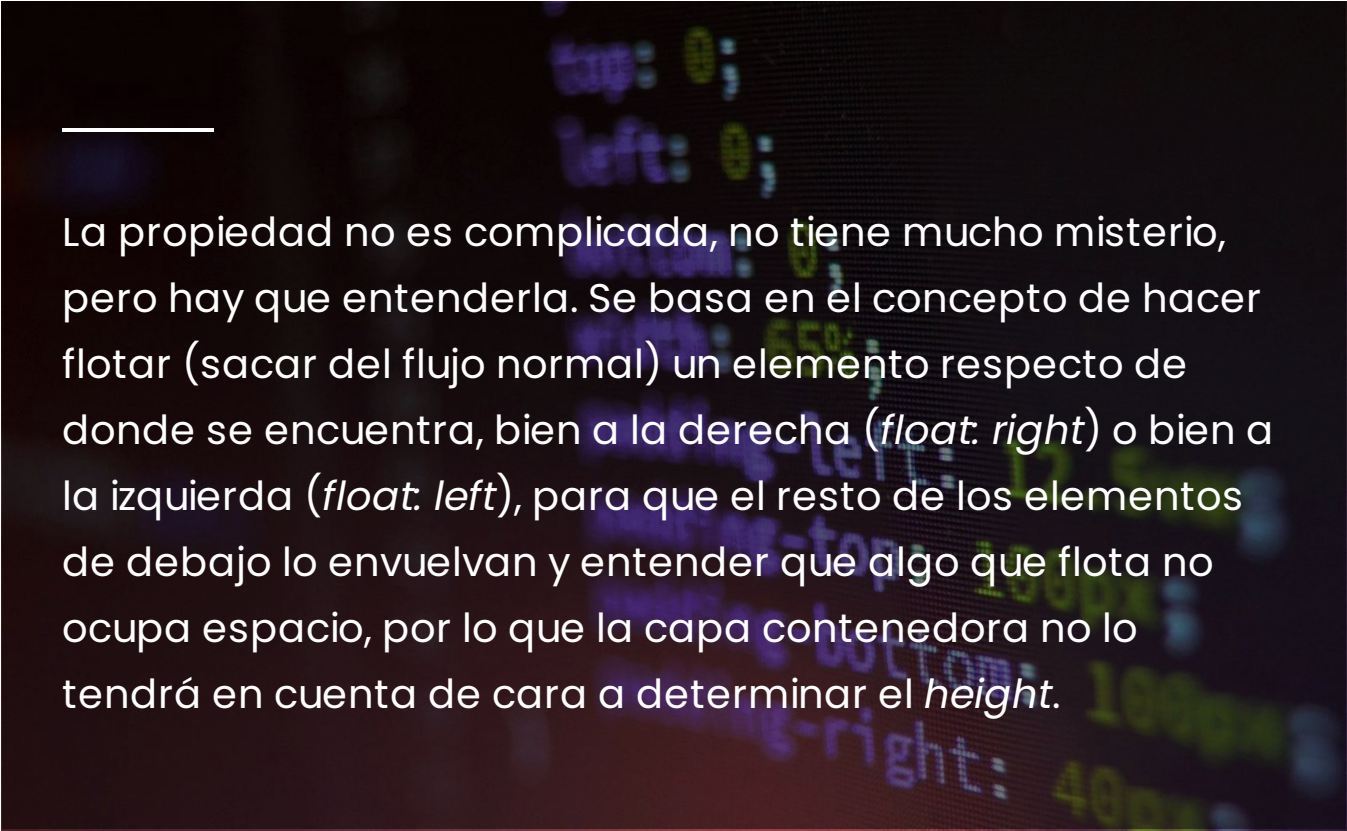


Fuente: <https://web.dev/i18n/es/learn/css/layout/>.

Float

Actualmente y en el pasado, esta propiedad se orienta a **la colocación de imágenes de manera flotante**, al igual que podemos hacer en un documento de Word posicionando delante, detrás o a los lados de un texto.

Pero no hace mucho tiempo, antes de la llegada del resto de estrategias de layout, no quedaba otra que utilizar esta propiedad conjuntamente con el position para la colocación estructural, para la **creación de columnas**. Es algo tedioso y reconozco que para todos la llegada de Flexbox supuso un alivio, ya que esta propiedad puede dar muchos quebraderos de cabeza incluso dominándola.



La propiedad no es complicada, no tiene mucho misterio, pero hay que entenderla. Se basa en el concepto de hacer flotar (sacar del flujo normal) un elemento respecto de donde se encuentra, bien a la derecha (*float: right*) o bien a la izquierda (*float: left*), para que el resto de los elementos de debajo lo envuelvan y entender que algo que flota no ocupa espacio, por lo que la capa contenedora no lo tendrá en cuenta de cara a determinar el *height*.

Todo flotará alrededor del elemento hasta que **se limpien los floats** a través de la propiedad *clear*, pudiendo limpiar los flotados a la derecha (*clear: right*), a la izquierda (*clear: left*) o ambos (*clear: both*).

Si quieres crear una distribución en dos columnas, tendrás que flotar una capa a la izquierda, otra a la derecha y, posteriormente, limpiar los floats. Para este caso, lo mejor que puedes hacer es englobar todos los elementos que quieras flotar en un elemento que ejerza de *wrapper* y al que le apliques un *hack*:

```
.float-left {  
  float: left;  
}  
  
.float-right {  
  float: right;  
}  
  
.wrapper::after {  
  content: "";  
  display: block;  
  clear: both;  
}
```

Como puedes ver en el ejemplo, he creado una clase para flotar a la izquierda, una clase para flotar a la derecha y la clase *wrapper* que aplica el *hack*. Lo que va a hacer este *hack* es limpiar todos los floats que existan justo después de la clase *wrapper*.

Te invito a que practiques a crear columnas y te pelees un poco con los problemas hasta que asimiles el funcionamiento. Y recuerda que ya no es necesario usarlo más allá de colocar imágenes en el texto: el layout constrúyelo con las propiedades que vamos a estudiar.

Flexbox

2012 se convirtió en un antes y un después para los maquettadores con la llegada de [Flexbox](#), una maravilla de propiedad que permitía **estructurar el contenido** sin quebraderos de cabeza.

Elimina todos los problemas ocasionados por la propiedad *float* y añade un nuevo valor a la propiedad ya existente *display*:

```
.row {  
  display: flex;  
  flex-wrap: wrap;  
  margin: 0 auto;  
  width: 100%;  
}
```

Con esta propiedad *display: flex* lo que se consigue es **autocontener el flotado de los elementos** (ítems) dentro de un bloque y permite distribuirlos de una forma mucho más sencilla. Permite, por ejemplo:

flex-direction: row | row-reverse | column | column-reverse —

Indicar si se quieren distribuir en forma de columna o en forma de fila, incluso de forma invertida al orden natural.

order —

Decidir el orden de los elementos usando la propiedad en un ítem.

flex-grow —

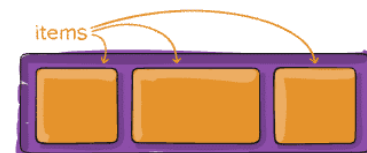
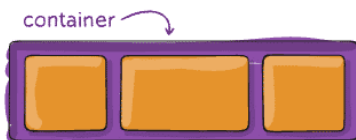
Indicar la proporción del espacio que ocupa cada ítem con la propiedad dentro de un ítem.

flex-wrap —

Determinar si se quiere que se adapten los ítems en tamaño a una única línea o, por el contrario, que caigan mágicamente cuando lo necesiten.

justify-content & align-items & align-content —

Distribuir vertical y horizontalmente los elementos y el contenido dentro del contenedor e, incluso, a un elemento en concreto con *align-self*.



Grid

Viajamos 5 años más y llegamos a 2017, aparece CSS Grid como nueva técnica para crear layouts. Al igual que flexbox, CSS Grid viene a resolver algunas complicaciones que teníamos con flexbox ya que en algunas ocasiones podía ser algo más compleja la estructuración del contenido.

CSS Grid nos permite definir una cuadrícula, como su propio nombre indica, para organizar los elementos de forma sencilla.

Podemos, desde el elemento contenedor, definir la cantidad de columnas y filas que queremos e, incluso, el tamaño que tiene que ocupar cada una de ellas, pudiendo utilizar diferentes unidades de medida y proporción. Esta es la gran diferencia que nos encontramos con respecto a Flexbox, que es algo más tedioso de hacer.

Al igual que con Flexbox, podemos organizar los elementos y el contenido tanto horizontal, vertical y variaciones.



Te recomiendo que hagas una lectura de [este artículo](#) en profundidad, ya que se hace una descripción con representación gráfica acompañada de todas las propiedades y sus valores. Estoy seguro de que te servirá de mucha ayuda. [Aquí](#), además, puedes encontrar una guía de las propiedades que nos aporta CSS Grid para estructurar nuestro contenido. Revísala, es necesario para tu formación.

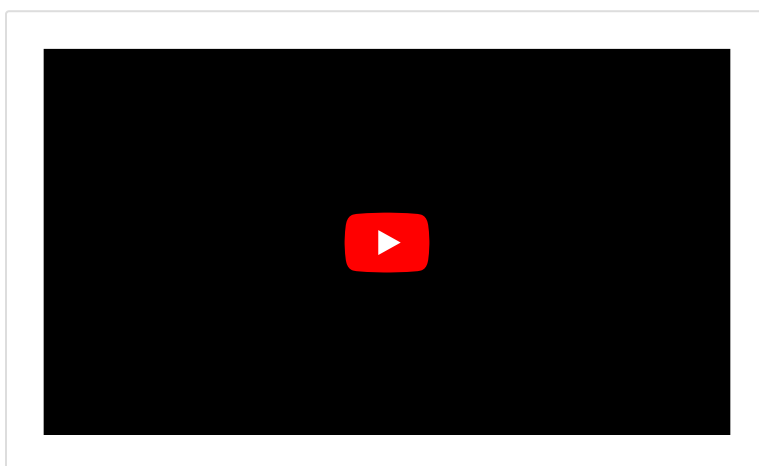
Pero, sobre todo, como en Flexbox, lo más importante es definir la propiedad `display`, que puede ser `grid` o `inline-grid` para que el elemento que estamos estilando se comporte como un grid, tanto en formato bloque como en formato en línea. Después de esto, ya podemos entrar a concretar algunas de las propiedades principales. ¡Vamos a verlas!

- *grid-template-columns*, *grid-template-rows* y *grid-template-areas* definen la cantidad y tamaño de columnas y filas con diferentes tipos de unidades como *px*, *%*, *fr* (fracciones de tamaño), *auto* (tamaño adaptativo), funciones (*repeat*, por ejemplo, que permite repetir 'n' veces la dimensión elegida), etc.
- *column-gap*, *row-gap* y *gap* permiten definir separaciones entre columnas y filas.
- *grid-column-** para colocar ítems específicos en columnas y filas.
- Y todas las propiedades que ya hemos visto en Flexbox y en el artículo anterior para alineamiento de los ítems.

Te dejo algunos **recursos interesantes** para amplíes tu aprendizaje:

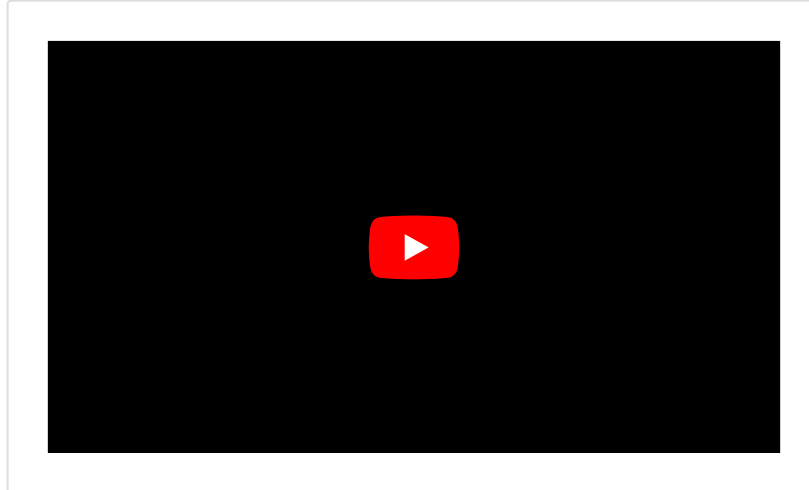
1

Un vídeo en el que **se explica CSS Grid**:



2

Un vídeo en el que se explica la **diferencia entre Flexbox y CSS Grid** y en qué casos se recomienda utilizar cada uno de ellos:



¿Qué hemos aprendido?

Ahora ya sabemos cómo podemos hacer **un sitio web atractivo** y darle la estructura adecuada, puesto que disponemos de los conocimientos suficientes como para poder construir sitios web con cierto nivel de complejidad.

No olvides aplicar correctamente las unidades, las variables, propiedades, selectores, etc., para que tanto **el entendimiento como el mantenimiento** de tus estilos sean una tarea sencilla.

Ahora toca asimilar todo lo contenido en este fastbook y practicar mucho para entender bien todo lo que hemos visto. Ya sabes cuál es el consejo que mejor te puedo dar... Practica, practica, practica ¡y practica! Solo así te convertirás en un maestro de la maquetación.

¡Enhorabuena! Fastbook superado